DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

Python project structure for bundles

Section 1. Modular code, deployment automation, and where tests actually live.

SECTION 1 of 10 WEIGHT 22% TOPIC Bundles

admin@sthithapragna.com

A bundle turns your whole project into one deployable, version-controlled unit. The exam tests what that buys you, and what it validates.

Why the notebook-first habit breaks

- Logic lives inside notebooks, so it cannot be imported, unit tested, or reused by a second job.
- Promoting to production means **editing paths and cluster IDs by hand**, in the workspace UI.
- Two engineers deploying at once overwrite each other's jobs. Nothing is namespaced.
- A bundle fixes all three by making the project a **source-controlled artefact**, not a set of workspace objects.

> ON THE EXAM
>
> The exam frames this as scale. If an answer involves manually copying notebooks between workspaces, it is almost certainly wrong.

What the default Python template generates

PATH	WHAT IT IS FOR
databricks.yml	The bundle definition. Name, includes, artifacts, variables, targets.
resources/	One file per job or pipeline. Pulled in by a glob.
src/	Your importable Python package. The code that is actually tested.
tests/	Unit tests, run locally or as part of the build.
pyproject.toml	Package metadata and the wheel build definition.
README.md	Generated. How to use this particular bundle.

ON THE EXAM

Created by 'databricks bundle init' with the default-python template. Prompts ask whether to include a notebook job, an ETL pipeline, a stub package and serverless.

What makes this structure scalable

- Logic sits in an **importable package** under the source folder, so a notebook and a job can both call it.
- Each job gets its **own resource file**. Two people editing two jobs never touch the same file.
- Tests import the package directly and run on a laptop, with no cluster and no workspace.
- Environment differences live in **targets**, not in the code. The code never knows which environment it is in.

> ON THE EXAM
>
> Modular development, deployment automation and continuous delivery are the three words the exam guide uses. This slide is all three.

Top-level mappings you should recognise

MAPPING	WHAT IT DOES
bundle	Name, and deployment settings such as the lock.
include	Globs that pull in resource files, typically resources/*.yml.
variables	Named values, referenced elsewhere as ${var.name}.
artifacts	How to build a wheel or JAR before deploying it.
resources	Jobs, pipelines, schemas, volumes and more.
targets	Named environments. Each can override almost anything.
workspace	Host, and the paths artifacts and files are written to.
run_as	The identity a deployed workflow runs under.

ON THE EXAM

Anything set inside a target overrides the same setting at the top level. That override rule is the whole reason one file can serve every environment.

Notebook task or Python wheel task

NOTEBOOK TASK

Fast, but hard to share

- Code and output live together
- Cannot be imported by another job
- Testing means running the notebook
- Cannot take a task environment spec

PYTHON WHEEL TASK

Built, versioned, importable

- Built from your source package
- One entry point, called by name
- Unit tested without a cluster
- Same artefact promoted to every target

WORTH REMEMBERING

A wheel task names a package and an entry point. That entry point is a function declared in the project file, not a file path.

The artifacts mapping, field by field

FIELD	MEANING
`type`	whl or jar. Required for a Python wheel.
`build`	The command to run. Defaults to python setup.py bdist_wheel.
`path`	Where the build runs. Defaults to the bundle root.
`dynamic_version`	Patch the wheel version from its file timestamp.
`files`	Paths to the built artefact files.
`executable`	bash, sh or cmd.

ON THE EXAM

Omit the artifacts mapping entirely and the CLI assumes the wheel is already built, deploying whatever the libraries whl entries point at.

Why yesterday's code keeps running

- You change the code, redeploy, and the job runs the **old behaviour**. Nothing errored.
- Compute caches an installed wheel **by version**. Same version number means no reinstall.
- Set **dynamic_version to true** and the version is patched from the file timestamp on every build.
- New code then deploys without anyone editing a version number in the project file.

> ON THE EXAM
>
> If a question describes redeployed code that behaves as though it never changed, the version number is the thing that did not move.

The command line cycle

01

init

Generate a project from a template, default-python or your own.

02

validate

Check the configuration parses and resolves. Catches errors before any deploy.

03

deploy

Build artefacts, upload files, create or update every resource in the target.

04

run

Trigger a named job or pipeline in that target and return a run URL.

05

summary

Show what is currently deployed. Use destroy to remove it.

ON THE EXAM

Every one of these takes a target. Forgetting which target you are on is the most common cause of a surprise in the wrong workspace.

Development mode against production mode

MODE: DEVELOPMENT

Built for iteration

- Prefixes resources with [dev yourname]
- Pauses all schedules and triggers
- Allows concurrent runs
- Disables the deployment lock
- Lets you override the cluster

MODE: PRODUCTION

Built for safety

- No prefix. Real names, real schedules
- Validates pipelines are not in dev
- Validates the Git branch matches
- Expects a service principal
- Refuses cluster overrides

OFTEN MISSED

Modes are optional. They are a convenience that applies a group of settings at once, not something a bundle requires.

What it validates before it will deploy

- Every declarative pipeline is marked **development: false**.
- The current Git branch matches the branch named in the target. Override with **--force**.
- Databricks recommends **run_as set to a service principal** for production.
- Without one: paths must not be overridden to a specific user, and run_as and permissions must be set.
- Cluster definitions **cannot** be overridden. No compute override is accepted.

> ON THE EXAM
>
> Deployments fail these checks far more often than they fail on syntax. Read the failure message, not the YAML.

Presets, and which setting wins

- Presets adjust mode behaviour: **name_prefix**, **trigger_pause_status**, **jobs_max_concurrent_runs**, **pipelines_development**, **tags**.
- Precedence runs: an **individual resource** beats a **preset**, which beats the **mode default**.
- So a job with max concurrent runs of one stays at one, even if the preset says twenty.
- And a schedule explicitly set to unpaused stays unpaused, even where the preset pauses everything.

> ON THE EXAM
>
> Most specific wins. When a question sets the same thing in two places, find the narrower one.

What the workspace looks like after a deploy

THING	WHERE IT GOES
Your files	Under the bundle root path for the target.
The built wheel	An internal artifacts folder beneath that root.
Jobs and pipelines	Created as real workspace objects, named per the mode.
Deployment state	Tracked by the bundle. This is how updates replace rather than duplicate.

ON THE EXAM

Set workspace.artifact_path to a Unity Catalog volume to keep wheels and JARs governed rather than loose in the workspace.

Where third-party libraries attach

- Classic compute: a **libraries** list on the task, pointing at a wheel path or a PyPI name.
- Serverless: an **environments** block with an environment key, a version, and a dependency list.
- A **notebook task cannot take an environment spec**. Use a notebook-scoped install instead.
- Tasks reference an environment by key, so several tasks can share one definition.

> ON THE EXAM
>
> The next module goes deeper on dependency resolution. What matters here is that the attachment point differs by compute type.

Where tests fit in this structure

- Tests import your package from the source folder and run locally. No cluster, no workspace.
- The **build command can run the tests first**, so a failing test blocks the artefact from being built.
- That makes the deploy step the enforcement point, rather than relying on discipline.
- Integration tests run afterwards, against a deployed development target.

> ON THE EXAM
>
> A structure that cannot be tested locally has failed the objective, whatever else it does well.

Bundles defined in Python

- Resources can be declared in **Python** instead of YAML, using the databricks-bundles package.
- A python section in the bundle file names a virtual environment and a loader function.
- Useful when jobs are generated from data, rather than written out one by one.
- It does not replace YAML. Both can coexist in one bundle, and YAML remains the default.

> ON THE EXAM
>
> Recognise it if it appears in an option. The exam's centre of gravity is still the YAML form.

Variables and substitutions

REFERENCE	RESOLVES TO
`${var.catalog}`	A variable you declared, overridden per target.
`${bundle.target}`	The name of the target being deployed.
`${workspace.current_user.short_name}`	The deploying user. This is what the dev prefix uses.
`${workspace.root_path}`	Where this bundle's files live for this target.
`${bundle.git.commit}`	The commit being deployed. Useful in artefact paths.

ON THE EXAM

Substitutions are why one configuration file serves every environment. Hard-coded catalog names in an answer are a red flag.

The prefix protects jobs, not tables

- The dev prefix separates **jobs**. It does nothing at all about where those jobs write.
- Two engineers can still write to the same table, and silently corrupt each other's results.
- The template offers a **personal schema per user** for exactly this reason.
- Catalog and schema become variables, overridden in the dev target to something user-specific.

> ON THE EXAM
>
> Compute isolation and data isolation are separate problems. A question about two developers interfering is usually about the second.

Git folders or bundles

GIT FOLDERS

Source control in the workspace

- Clone a repo into the workspace
- Edit and commit from the UI
- Good for interactive development
- Does not create jobs or pipelines

BUNDLES

Deployment of the whole project

- Defines jobs, pipelines and permissions
- Deploys from a laptop or a pipeline
- Targets carry environment differences
- Does not give you an editing surface

NOT EITHER-OR

They are complements, not alternatives. Git folders are where a human edits; bundles are how the project reaches an environment.

sync and permissions

- **sync** controls which local files are uploaded. Include and exclude patterns.
- Useful for keeping test fixtures, notebooks or large data out of the workspace.
- **permissions** grants access on the deployed resources, declared alongside them.
- Declaring permissions in the bundle means access is **reviewed in version control**, not clicked in a UI.

> ON THE EXAM
>
> Production mode expects permissions to be stated explicitly when you are not deploying as a service principal.

What to carry into the exam

- Logic in an importable package. Jobs in their own resource files. Environment differences in targets.
- **Development mode** prefixes, pauses schedules, and permits cluster overrides.
- **Production mode** validates pipelines, branch, run identity and permissions, and forbids cluster overrides.
- Most specific setting wins: resource over preset over mode.
- Redeployed code behaving unchanged means the wheel version did not move.

> ON THE EXAM
>
> If an answer requires a person to edit something in the workspace to promote a change, it is the wrong answer.

A team's bundle deploys cleanly to their dev target. The same command against the prod target fails immediately, before anything is created.

The prod target sets mode to production and names a Git branch of main. The engineer is working on a feature branch, and has also passed a flag pointing the deployment at an existing cluster to save start-up time.

What is the most likely cause of the failure?

A. The bundle is missing an artifacts mapping, so no wheel could be built.

B. Production mode validated the Git branch against the target, and refused the cluster override.

C. Production mode requires a service principal, so deployment always fails without one.

D. The prod target inherits the dev prefix, colliding with existing production job names.

◆ PAUSE AND DECIDE

What is the most likely cause of the failure?

A The bundle is missing an artifacts mapping, so no wheel could be built.

Missing artifacts does not fail. The CLI assumes the wheel is already built.

B Production mode validated the Git branch against the target, and refused the cluster override.

Both checks are tripped at once. Either alone would stop the deploy.

C Production mode requires a service principal, so deployment always fails without one.

Recommended, not mandatory. Without one you get extra validations, not a hard stop.

D The prod target inherits the dev prefix, colliding with existing production job names.

The prefix is a development mode behaviour. Production mode never applies it.

ON THE EXAM

Read the failure message before the configuration. Production deployments fail validations far more often than syntax.

A data engineer fixes a rounding bug in a shared transformation function, commits it, and redeploys the bundle to the development target. The deployment succeeds with no warnings.

The wheel task runs, completes successfully, and the output still shows the old rounding. Nothing in the logs indicates an error, and the fix is definitely present in the committed source.

Which change is most likely to resolve this?

A. Set dynamic_version to true so the wheel version changes on every build.

B. Switch the wheel task to a notebook task so code is read fresh on each run.

C. Add the transformation module to the job's libraries list as well as the wheel.

D. Run bundle destroy, then bundle deploy, to force a clean workspace state.

◆ PAUSE AND DECIDE

Which change is most likely to resolve this?

A Set dynamic_version to true so the wheel version changes on every build.

Compute caches an installed package by version. Patching it from the build timestamp forces a reinstall.

B Switch the wheel task to a notebook task so code is read fresh on each run.

It would work, which is what makes it dangerous. It trades away testability and reuse for a one-line setting.

C Add the transformation module to the job's libraries list as well as the wheel.

Listing it twice does not change the version, so the cached wheel still wins.

D Run bundle destroy, then bundle deploy, to force a clean workspace state.

Fixes it once by accident. The same bug returns on the very next change.

ON THE EXAM

Redeployed code that behaves as though it never changed is a version problem, not a deployment problem.

> Two engineers work on the same bundle in the same workspace. Both deploy to a target with mode set to development. The job definition in the repository specifies a nightly schedule.
>
> Neither wants their in-progress work running on a schedule. One engineer additionally needs their own copy to run on a schedule for a latency test.

Which statement describes the resulting behaviour?

A. Both deployments overwrite the same job; the nightly schedule runs for whoever deployed last.

B. Both get separately named jobs with schedules paused; the second sets pause_status to unpaused on their own job.

C. Both get separately named jobs, but schedules stay active because pausing needs an explicit preset.

D. Development mode blocks deployment of any job carrying a schedule.

◆ PAUSE AND DECIDE

Which statement describes the resulting behaviour?

A Both deployments overwrite the same job; the nightly schedule runs for whoever deployed last.

Overwriting is precisely what the dev prefix exists to prevent.

B Both get separately named jobs with schedules paused; the second sets pause_status to unpaused on their own job.

The prefix carries the user's short name, and an individual resource setting beats the mode default.

C Both get separately named jobs, but schedules stay active because pausing needs an explicit preset.

Inverted. Pausing is already what development mode does.

D Development mode blocks deployment of any job carrying a schedule.

No such restriction exists.

ON THE EXAM

Development mode gives you isolation and paused schedules by default, and both can be overridden per resource.

A team is restructuring a project currently made of six notebooks that duplicate the same cleansing logic.

They want the logic unit tested on a laptop, reused by both a scheduled job and an interactive notebook, promoted unchanged from development to production, and they want a failing test to prevent a release.

Which approach best meets all four requirements?

A. Keep the notebooks in Git folders and run them directly as tasks in each environment.

B. Move the logic into a source package, build it as a wheel, and run the tests in the build command.

C. Move the logic into one shared notebook and call it from the others.

D. Define every job in Python with the bundles package, keeping the cleansing logic in each job definition.

◆ PAUSE AND DECIDE

Which approach best meets all four requirements?

A Keep the notebooks in Git folders and run them directly as tasks in each environment.

Versions notebooks, but does not make them importable or locally testable.

B Move the logic into a source package, build it as a wheel, and run the tests in the build command.

Importable means testable locally and callable from both. Tests in the build block the artefact.

C Move the logic into one shared notebook and call it from the others.

Removes duplication and solves nothing else. It still needs a cluster to run at all.

D Define every job in Python with the bundles package, keeping the cleansing logic in each job definition.

Python-defined bundles describe resources. Business logic does not belong in a job definition.

ON THE EXAM

Four requirements, and only one option meets all four. Check each requirement against each option rather than picking on feel.

> A bundle writes to a table named explicitly as main.analytics.orders inside the job definition. It deploys to a development target with mode set to development, and two engineers use it daily.
>
> Both engineers report that their test runs are producing results they cannot explain, with rows appearing that neither of them created. Each has confirmed their own job runs successfully.

What is the underlying problem?

A The development prefix is not being applied, so both engineers share one job.

B The table name is hard-coded, so both isolated jobs write to the same table.

C The deployment lock is disabled in development mode, corrupting concurrent deploys.

D Concurrent runs are enabled, so each engineer's job runs twice.

◆ PAUSE AND DECIDE

What is the underlying problem?

A The development prefix is not being applied, so both engineers share one job.

The prefix is automatic in development mode, and both report their own job running.

B The table name is hard-coded, so both isolated jobs write to the same table.

Development mode isolates jobs, not data. Only variables for catalog and schema fix this.

C The deployment lock is disabled in development mode, corrupting concurrent deploys.

The lock governs deploys, not table writes.

D Concurrent runs are enabled, so each engineer's job runs twice.

Concurrent runs would duplicate work, not introduce another person's rows.

ON THE EXAM

Isolated compute with shared data is a classic trap. Look at where the job writes, not just what it is called.

> A team wants analysts to explore and edit shared notebooks directly in the workspace with full version history. Separately, they want their nightly production jobs created and updated automatically from the same repository, with no manual steps.
>
> They are debating whether to adopt Git folders or bundles.

Which approach meets both needs?

A Git folders only, running the notebooks in the folder as scheduled tasks.

B Bundles only, with analysts editing the deployed files in the workspace.

C Git folders for interactive editing, and bundles to deploy the jobs.

D Neither. Use the Jobs UI and export definitions to the repository afterwards.

◆ PAUSE AND DECIDE

Which approach meets both needs?

A Git folders only, running the notebooks in the folder as scheduled tasks.

Covers the editing need and leaves job creation manual.

B Bundles only, with analysts editing the deployed files in the workspace.

Deployed files are outputs of a deploy. Editing them there puts changes outside version control.

C Git folders for interactive editing, and bundles to deploy the jobs.

They solve different problems and are designed to be used together.

D Neither. Use the Jobs UI and export definitions to the repository afterwards.

Exporting after the fact makes the workspace the source of truth, which is the thing to avoid.

ON THE EXAM

When two requirements point at two tools, the answer is usually both. Watch for options that force an unnecessary choice.